

Event-Driven Music Recommendation Systems: A Technical Case Study of Spotify-Style Recommendation Architecture Using Apache Kafka, Big Data, and Machine Learning

Jyatin

Department of Computer Science and Engineering
Lovely Professional University, Punjab, India

Technical Case-Study / Research Paper — September 2026

Abstract—Music-streaming platforms must translate an unending stream of short, low-signal user actions — plays, skips, saves, and searches — into personalized recommendations that remain accurate as catalogs and audiences scale into the hundreds of millions. This case study examines how an event-driven architecture built around Apache Kafka, complemented by distributed batch processing and machine-learned ranking, can address that problem. Rather than describing any single company's internal system, the paper develops an original reference architecture that separates event ingestion, stream processing, candidate generation, ranking, and re-ranking into distinct, independently scalable stages, and it discusses how an online (streaming) path and an offline (batch) machine-learning path can share a common event log without duplicating business logic. Spotify is used throughout as a motivating, publicly documented example of the class of problem being addressed, but the paper is explicit that the proposed architecture is a reference design informed by public engineering discussion and by peer-reviewed recommender-systems research, not a description of Spotify's proprietary internals. The paper works through the underlying mechanisms in Kafka — partitioning, consumer groups, replication, and replayability — and connects them to recommendation concepts including implicit-feedback collaborative filtering, embedding-based candidate retrieval, and learning-to-rank. A concise mathematical formulation frames the recommendation task as a constrained scoring problem that trades off relevance, diversity, novelty, and freshness under latency constraints. The paper closes with an evaluation methodology built on standard offline ranking metrics, a discussion of scalability, reliability, and bias-related challenges, and an assessment of what would be required to move from this reference design toward a fully implemented, empirically evaluated system.

Index Terms—*event-driven architecture, Apache Kafka, recommender systems, collaborative filtering, stream processing, big data, music streaming, learning-to-rank.*

I. INTRODUCTION

Recommendation has become the primary interface through which listeners encounter music on streaming platforms: catalogs now number in the tens of millions of tracks, and most listening sessions begin from an algorithmically assembled surface rather than a deliberate search. This shift places unusual demands on the underlying systems architecture. A recommendation is only useful if it reflects what a listener wants now, which means the system must absorb continuous behavioural signals — plays, skips, completions, saves, and playlist edits — and let those signals influence future recommendations within a short window, while simultaneously drawing on months or years of historical listening to model long-term taste. Meeting both requirements at once is

fundamentally an architectural problem as much as a modelling one.

Traditional batch-oriented data pipelines, in which interaction logs are periodically extracted, transformed, and loaded into a warehouse for offline model training, handle the long-term signal well but are structurally unable to reflect what a listener did a few seconds ago. Purely synchronous, request-response architectures handle immediacy but struggle to decouple the many downstream consumers — analytics, feature computation, model training, experimentation — that all need to observe the same interaction events without coupling their availability to the primary application path. Event-driven architectures, in which every interaction is published as an immutable, ordered event to a durable log and consumed independently by any number of downstream systems, offer a way to reconcile these requirements. Apache Kafka has become a de facto standard implementation of this pattern in industry, precisely because it treats the event log itself, rather than any single consumer, as the durable source of truth [1], [2].

This paper undertakes a technical case study of how such an event-driven architecture, combined with distributed batch processing and machine-learned ranking, can support scalable, fresh, and low-latency personalized music recommendation. Spotify is used as the motivating real-world example throughout, because its Discover Weekly feature is among the most widely discussed applications of large-scale music recommendation and has been the subject of public conference talks, engineering interviews, and independent technical analysis. It is important to state plainly, however, that this paper does not claim to reproduce Spotify's actual internal architecture. Spotify has not published a complete technical specification of its recommendation infrastructure in the peer-reviewed literature, and the publicly available material — executive interviews, engineer commentary, and third-party technical write-ups — describes the high-level ideas involved (collaborative filtering, audio content models, and natural-language analysis of playlist and press context) without disclosing implementation detail [3], [4], [5]. Throughout this paper, four categories of material are kept explicitly distinct: (i) information that is publicly documented about Spotify's approach, drawn from named, attributable sources; (ii) established results from the recommender-systems and distributed-systems literature; (iii) an original reference architecture proposed by this paper, which is illustrative rather than a claim about any deployed system; and (iv) the paper's own analytical discussion and proposed evaluation methodology.

A. Research Questions

The paper is organized around one central research question and eight subsidiary questions that decompose it into the

architectural and algorithmic decisions a real system would need to make.

Central question: How can an event-driven architecture based on Apache Kafka, distributed big-data processing, and machine-learning recommendation models support scalable, fresh, and low-latency personalized music recommendations?

The subsidiary questions addressed in Sections III through VI are: (1) how continuous listening events should be captured and processed; (2) why Kafka in particular is useful as the event backbone; (3) how streaming and batch processing can complement rather than duplicate one another; (4) how short-term (session-level) and long-term (taste-level) user preferences can be represented jointly; (5) how candidate generation, ranking, and re-ranking should be separated as distinct stages; (6) what the major scalability and reliability challenges are; (7) how recommendation quality should be evaluated in the absence of ground-truth preference labels; and (8) what trade-offs exist between freshness, relevance, diversity, latency, and computational cost.

B. Contributions and Paper Organization

The contribution of this paper is a synthesis rather than a novel algorithm: it connects well-established ideas from event-streaming infrastructure and from the recommender-systems literature into a single, coherent reference architecture for music recommendation, argues for specific design choices at each stage, and proposes an evaluation methodology appropriate for a system of this kind. Section II reviews the necessary background in Kafka, big-data processing, and recommender systems. Section III discusses what is and is not publicly known about Spotify's own approach, to keep that material clearly separated from the paper's own proposal. Section IV presents the proposed reference architecture, including the five architectural diagrams and the event-and-feature tables required to make the design concrete. Section V gives a mathematical formulation of the ranking and re-ranking problem. Section VI proposes an evaluation methodology built on standard offline metrics. Section VII discusses scalability, reliability, and bias-related trade-offs. Section VIII states the paper's limitations, and Section IX concludes.

II. BACKGROUND AND RELATED WORK

A. Apache Kafka as an Event Backbone

Apache Kafka was originally developed at LinkedIn to solve a specific infrastructural problem: consumer internet companies generate very high volumes of “log” data — user activity events, operational metrics, and system telemetry — that need to reach multiple downstream systems with low latency, and existing enterprise messaging systems were not designed for this throughput, while existing log aggregators were designed only for offline consumption [1]. Kafka's design choices follow directly from that problem statement, and several of them are central to the architecture proposed in this paper.

A Kafka topic is a named, append-only log of records, and every topic is divided into one or more partitions. Within a partition, Kafka guarantees a strict, immutable order of records, identified by a monotonically increasing offset; ordering across partitions of the same topic is not guaranteed, which is the trade Kafka makes to allow partitions to be spread across brokers and consumed in parallel [2]. Producers choose which partition a given record is written to, typically by hashing a key such as a user identifier, which has the useful side effect that all events

for a given user land in the same partition and are therefore consumed in the order they were produced. Consumers read partitions and track their own position (offset) in the log; because the log itself, not the consumer, retains this history for a configurable retention period, a consumer can be paused, restarted, or entirely replaced without losing data, and multiple independent consumers can replay the same events without interfering with one another [1], [2].

Consumers are organized into consumer groups. Kafka guarantees that each partition is consumed by exactly one member of a given consumer group at any point in time, which provides load-balanced, parallel consumption within a group, while separate consumer groups each receive an independent full copy of the stream — a topic can simultaneously feed a real-time feature generator, an analytics pipeline, and a data-lake writer, each reading at its own pace [2]. This is the property that most directly supports the dual online/offline architecture developed in Section IV: the same playback event is consumed once by the low-latency stream processor that updates session features and, independently, by the process that appends the event to long-term historical storage, without either consumer's throughput or availability affecting the other.

Durability follows from replication. Each partition has a designated leader broker that services all reads and writes for that partition, and a configurable number of follower brokers that replicate the leader's log; only followers that are caught up within a bounded lag are considered part of the in-sync replica set, and only members of that set are eligible for leader election if the current leader fails, which is what allows a single broker failure to occur without data loss or a change in the offsets already acknowledged to producers [2]. This reliability property matters for a recommendation pipeline specifically because interaction events double as training data: an event lost between the client and durable storage is not just a missed real-time update but a permanently missing row in future model training.

B. Big-Data Processing: Batch, Streaming, and Their Reconciliation

A recurring architectural question in large-scale data systems is how to reconcile a batch layer, which recomputes results from the complete historical record and is therefore accurate but slow, with a speed layer, which processes only recent events and is therefore fast but necessarily incomplete. The lambda architecture pattern, associated with Nathan Marz, addresses this by running both layers in parallel and merging their outputs at query time. In practice, this requires the same business logic to be implemented twice — once in a batch framework and once in a stream-processing framework — and keeping the two implementations behaviourally identical has proven to be a significant and recurring source of operational complexity [4].

Jay Kreps, one of Kafka's original authors, argued in a widely cited 2014 critique that because a durable, replayable log such as Kafka already retains the full history of events, “batch” reprocessing can be achieved simply by replaying the log through the same stream-processing job at higher parallelism, rather than maintaining a wholly separate batch codebase; this pattern is often referred to as the kappa architecture [4]. The reference architecture proposed in Section IV takes a middle position that is common in industrial recommendation systems: it retains a genuine batch path for computationally heavy operations that are impractical to run continuously — full

model retraining, large-scale feature backfills, and offline evaluation — while treating Kafka's retained log, rather than a separately maintained warehouse export, as the single source of truth that both the batch and streaming paths read from. This narrows, without fully eliminating, the dual-implementation problem that motivated the kappa critique.

C. Recommender Systems: From Collaborative Filtering to Learned Ranking

Recommender systems are broadly organized around two families of signal. Content-based approaches represent items using descriptive attributes — genre, tempo, instrumentation, or audio-derived features — and recommend items similar to those a user has previously engaged with. Collaborative filtering instead exploits regularities in the interaction matrix itself: users who behaved similarly in the past are assumed to share future preferences, without any need to understand why. Matrix factorization emerged from the Netflix Prize competition as the dominant collaborative-filtering technique, representing each user and each item as a low-dimensional latent vector such that their inner product approximates the user's affinity for the item; this approach was shown to outperform classical nearest-neighbour techniques while also accommodating implicit feedback, temporal dynamics, and varying confidence levels [5].

Music-streaming interaction data is overwhelmingly implicit: a play, a skip, or a completion indicates behaviour, not a stated preference, and negative examples are never directly observed — an unplayed track might be actively disliked or simply never surfaced. Hu, Koren, and Volinsky proposed a formulation for exactly this setting, treating implicit signals as indicators of confidence rather than direct preference labels and reweighting the factorization objective accordingly [6]. Rendle et al. proposed an alternative that optimizes directly for the relative ranking of observed versus unobserved items rather than for predicting an interaction score, an objective that tends to align more closely with the top-K retrieval task recommendation systems actually need to solve [7]. Later work extended the underlying similarity function from a simple inner product to a learned neural network, allowing more complex, non-linear interactions between user and item representations to be captured [8].

At the scale of a major streaming platform, however, no ranking model can be evaluated against every item in the catalog for every request within an acceptable latency budget. Industrial systems resolve this by splitting the problem into two stages, most influentially described in the YouTube recommendation system: a lightweight candidate-generation stage narrows a catalog of many millions of items down to a few hundred plausible candidates per request, typically using an embedding-based nearest-neighbour lookup, and only that small candidate set is passed to a more expensive ranking model that can afford to use richer features and a heavier model architecture [9]. Making that first-stage nearest-neighbour lookup fast enough to run online, over embedding collections at the scale of a modern catalog, is itself a systems problem; libraries such as FAISS demonstrate that approximate nearest-neighbour search over hundreds of millions to billions of vectors can be made to run within a small number of milliseconds using compressed indexing structures and GPU acceleration, at the cost of exact recall [10].

A persistent difficulty in all of these approaches is the cold-start problem: a new track or a new listener has no interaction history for collaborative methods to exploit, and a purely collaborative system will therefore systematically under-recommend recent releases. Content-based signals — audio features and metadata that are available at ingestion time, before any listener has interacted with a track — provide one mitigation, since they do not depend on accumulated interaction history. A related and more subtle problem is popularity bias: because collaborative-filtering objectives are typically trained to reproduce the interaction distribution in the training data, and that distribution is itself long-tailed, learned models tend to concentrate exposure on already-popular items even beyond what raw popularity would justify, which narrows the effective diversity of what listeners are shown and can be measured as a form of exposure bias across both listeners and the catalog of artists competing for attention [12].

III. SPOTIFY AS A MOTIVATING EXAMPLE

Spotify is used throughout this paper as the motivating real-world example because its Discover Weekly playlist, launched in 2015, is one of the most widely discussed consumer-facing applications of large-scale recommendation, and because several Spotify engineers and executives have spoken publicly, in interviews and conference presentations, about the categories of technique involved. It is necessary to be precise about the epistemic status of that material before using it to motivate an architecture.

What is publicly documented, from named and attributable sources, is the following. Spotify's chief executive has described Discover Weekly informally as combining a listener's own playlists, a modelled taste profile, and the catalog, generated from data analyzed at very large daily volume; this is an executive-level characterization rather than a technical specification. In a press interview, the engineer credited with building Discover Weekly described the system as resting on two complementary models: one that models relationships between tracks from the aggregate pattern of user-created playlists, and one that models each individual listener's taste from their own listening and follow behaviour, with the two combined to surface tracks a listener has not yet heard [3]. Independent, secondary technical write-ups — not Spotify engineering publications — describe the system as combining collaborative filtering over listening and playlist co-occurrence with content-based audio and natural-language signals, and note that collaborative signals alone cannot recommend newly released tracks with no listening history, which is why a content-based component is needed alongside it [4], [5].

What is not publicly documented, to this paper's knowledge, is any complete technical specification of Spotify's production infrastructure: the specific event schemas in use, whether or how Kafka or a comparable system is deployed internally, the exact candidate-generation and ranking model architectures, serving latency figures, or measured recommendation-quality metrics. No peer-reviewed publication by Spotify engineers describing the full recommendation pipeline was identified in the course of preparing this paper. Consequently, every specific figure, threshold, or infrastructural detail used later in this paper — partition counts, feature names, model choices — belongs to the proposed reference architecture in Section IV, which is this paper's own design exercise informed by the general engineering literature on event-driven systems and

recommendation, and should not be read as a claim about what Spotify or any other named company actually runs in production.

The remainder of this paper therefore treats Spotify strictly as a motivating example of the problem class — a catalog of tens of millions of tracks, hundreds of millions of listeners, and a product surface that depends on both long-term taste modelling and near-real-time responsiveness — and develops an architecture intended to be a plausible, technically coherent answer to that problem class in general, rather than a reconstruction of any specific company's system.

IV. PROPOSED REFERENCE ARCHITECTURE

This section presents an original reference architecture for a Spotify-style recommendation platform. It is organized as two cooperating paths that share a single event log: an online path that serves recommendations with low latency and reflects very recent behaviour, and an offline path that periodically retrains models against the full historical record. Figure 1 gives the high-level view; Figures 2 through 5 develop the ingestion layer, the request-level control flow, the offline/online machine-learning pipeline, and the ranking model internals in turn.

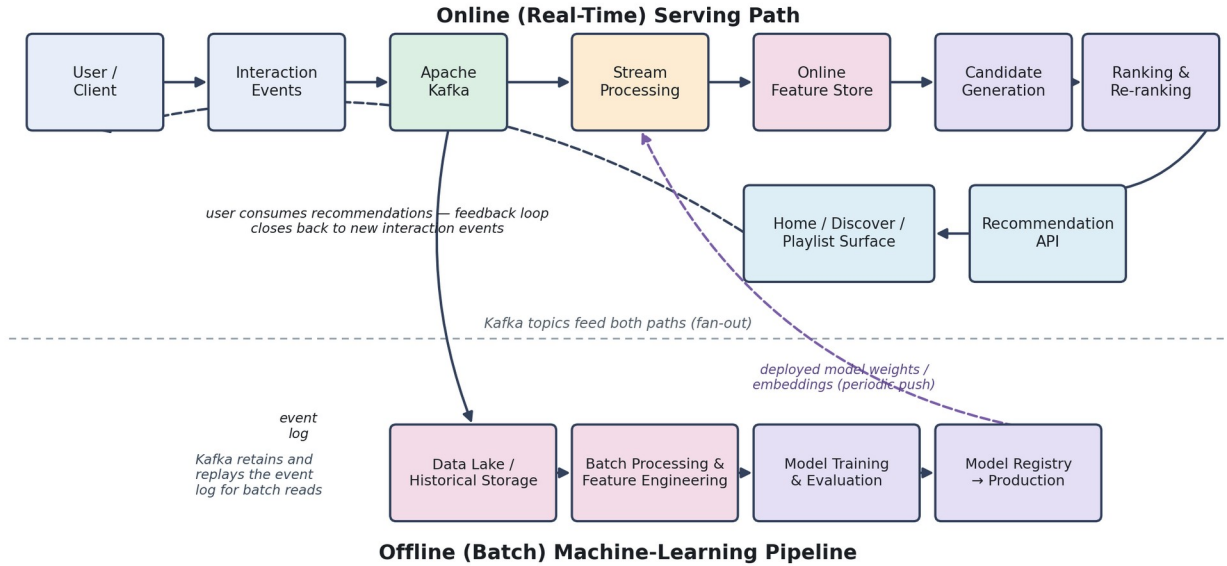


Fig. 1. High-level event-driven recommendation architecture. The online serving path (top) turns each interaction into a served recommendation with low latency; the offline machine-learning pipeline (bottom) periodically retrains the models that path depends on, reading from the same Kafka-retained event log.

A. Event Ingestion and the Kafka Layer

Every meaningful listener action — play, skip, save, search, playlist edit, and completion — is emitted as a structured event by the client or by an edge service immediately behind it, and published to a topic-partitioned Kafka cluster rather than written directly to any single downstream store. Figure 2 shows this layer in more detail. Separating events into distinct topics (playback-events, search-events, recommendation-impressions, save-events, session-events) allows each downstream consumer to subscribe only to the streams it needs, and allows the retention policy, partition count, and schema evolution of each stream to be tuned independently; recommendation-impression events, for instance, are needed for evaluation and exposure-bias analysis but are far higher-volume than save events and benefit from more aggressive compaction.

Partitioning each topic by a hashed user identifier keeps every event belonging to one listener in strict order within a single partition, which matters because session-level feature computation depends on the sequence of a listener's own actions, not on any global ordering across listeners. Two independent consumer groups read from these topics in the proposed design: one group, comprising the stream processor and the online feature generator, is optimized for low latency and shares partitions to balance load; a second group, comprising the analytics pipeline and the data-lake writer, reads the identical event stream on its own schedule, entirely decoupled from the first. Because Kafka retains the log rather than deleting events once one consumer has read them, this second group can lag, be reprocessed, or be replaced without any risk to the first.

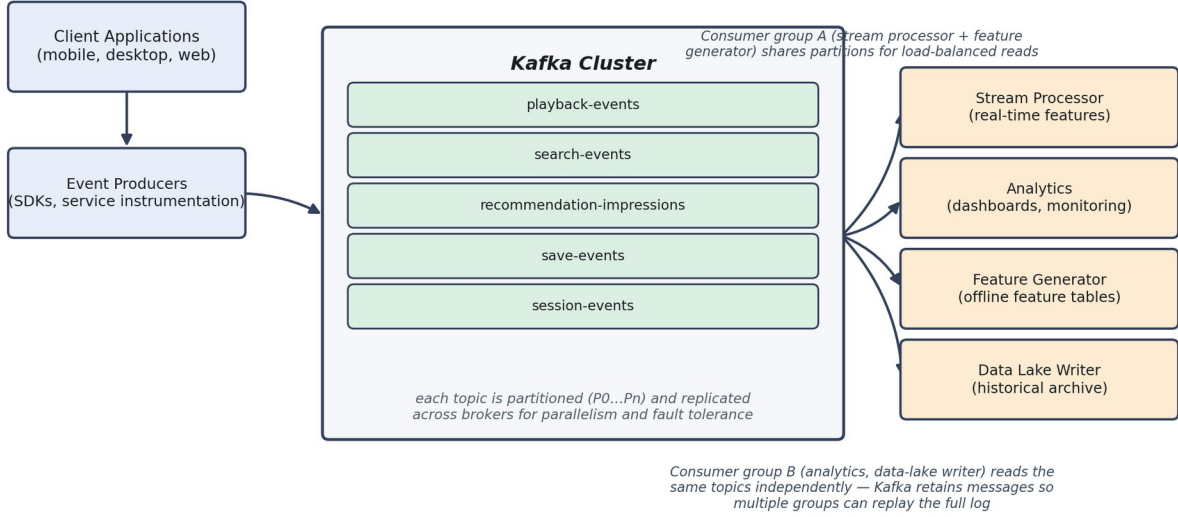


Fig. 2. Kafka-based event streaming pipeline. Producers publish to topic-partitioned streams; two independent consumer groups — a low-latency streaming group and a batch/analytics group — read the same events without interfering with each other.

B. Serving-Time Control Flow

Figure 3 traces what happens for a single request to a recommendation surface, and closes the loop back to event ingestion. When a listener opens a surface such as Home or a Discover-style playlist, the serving system retrieves the listener's current state (recent session features and a cached long-term taste representation), uses that state to generate a candidate set, fetches any additional features those candidates require, scores the candidates with the ranking model, re-ranks the scored list for diversity and business constraints, and returns the final list. Every resulting impression and any subsequent interaction — a play, a skip, a save — is itself published back to Kafka as a new event, which is what closes the feedback loop: a system's own recommendations become part of the training data for its next iteration, which is precisely why exposure bias (Section II-C) is a structural risk rather than an incidental one.

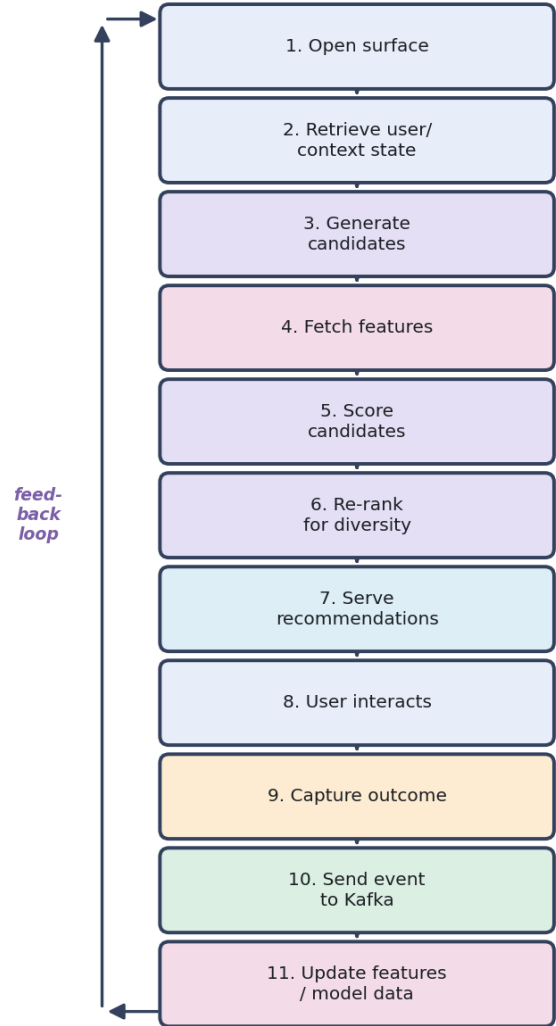


Fig. 3. End-to-end recommendation flow, numbered by stage. The feedback loop (left) returns newly generated interaction and impression events to Kafka, closing the cycle back to stage 1.

C. Candidate Generation, Ranking, and Re-Ranking as Separate Stages

Consistent with the two-stage design established for the YouTube recommendation system [9], the proposed architecture keeps candidate generation and ranking as separate components with different computational budgets. Candidate generation narrows a catalog of tens of millions of tracks to a few hundred plausible candidates per request by performing an approximate nearest-neighbour lookup over learned user and item embeddings, using an index structure of the kind implemented in libraries such as FAISS [10]; this stage optimizes for recall at the expense of ranking precision, and can pull from multiple independent sources in parallel — a collaborative-filtering embedding space, a content/audio embedding space for cold-start coverage, and simple popularity- or freshness-based candidate pools — whose outputs are merged before ranking.

The ranking stage receives only this small candidate set and can therefore afford a substantially more expensive model: a gradient-boosted tree or neural learning-to-rank model that consumes user, item, context, and session features jointly, of the kind made practical for implicit engagement data by the objective proposed by Rendle et al. [7] and extended to non-linear user-item interaction functions in later neural collaborative-filtering work [8]. A final re-ranking stage then adjusts the ranked list for considerations the ranking model does not directly optimize — catalog and artist diversity, novelty relative to what the listener has already been shown, and any product- or business-level constraints — before the list is returned through a recommendation API to the relevant surface. Fig. 6 sketches, purely illustratively, how a typical request's latency budget might be distributed across these stages.

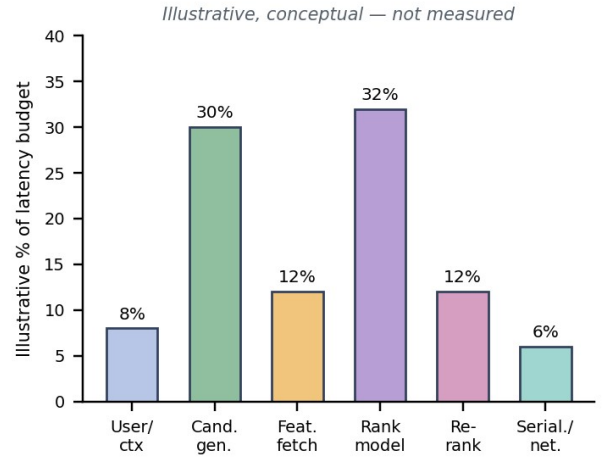


Fig. 6. Illustrative, conceptual breakdown of a typical request's latency budget across pipeline stages. This is a qualitative sketch to aid intuition, not a measurement from an implemented system.

D. Offline–Online Machine-Learning Pipeline

Figure 4 separates the two paths that jointly keep the ranking and candidate-generation models current. The offline path reads historical events from durable storage, performs cleaning and sessionization, engineers features over long time windows that would be impractical to maintain purely in a streaming context (for example, thirty-day genre-affinity profiles), assembles a training dataset, trains and evaluates candidate models offline, and promotes an approved model through a model registry into production. The online path computes lightweight, low-latency features directly from the streaming layer — the last few tracks played in the current session, whether the listener has skipped several tracks in a row — and writes them to an online feature store that the serving path reads from at request time. The two paths meet at two points: the production model trained offline is periodically pushed into the online scoring function, and features logged from the online store are later joined into the offline training dataset so that the model continues to learn from the same signals it is served with, which reduces the risk of training/serving skew.

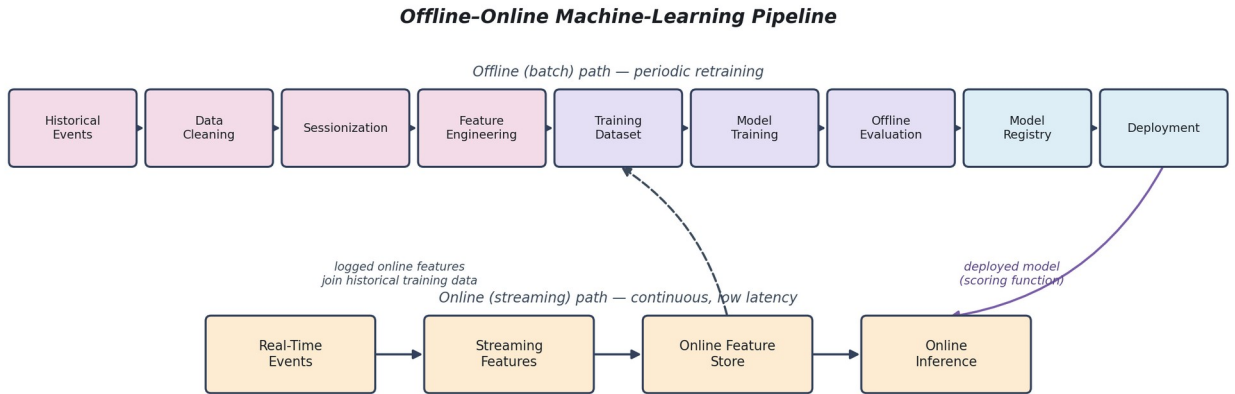


Fig. 4. Offline–online machine-learning pipeline. The offline (batch) path retrains models against the full historical record; the online (streaming) path computes low-latency features and serves inference.

E. Ranking Model Composition

Figure 5 shows the internal composition of the scoring stage used by both candidate retrieval and ranking. Four feature groups are combined: a user representation (a learned embedding summarizing long-term taste, typically produced by the offline matrix-factorization or neural collaborative-filtering process [5], [8]), an item representation (combining a collaborative embedding with content-derived audio and metadata features, which is what allows newly released tracks to be represented even before they accumulate interaction history), context features (time of day, device, and listening surface), and recent session features (the short sequence of actions taken in the current session). These are combined, used for approximate nearest-neighbour candidate retrieval, passed through the learning-to-rank model, and adjusted by the re-ranking layer to produce the final top-K list, formalized in Section V.

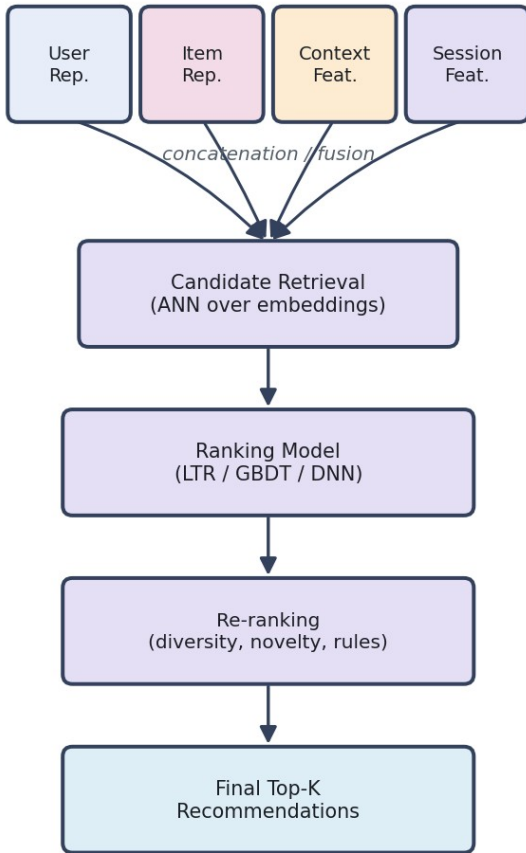


Fig. 5. Recommendation model architecture. User, item, context, and session features are fused, used to retrieve candidates, scored by a ranking model, and adjusted by a re-ranking layer.

F. Event Types and Derived Features

Table I compares the recommendation approaches introduced in Section II in terms of their suitability for different stages of the proposed architecture, and Table II maps the raw event types captured at ingestion to the features they can support downstream, illustrating why the event schema design in Section IV-A was organized around these particular topics.

G. Illustrative End-to-End Deployment View

Figures 1–5 present the architecture as a set of logical pipelines. Fig. 8 redraws the same design as a single illustrative deployment diagram, in the style of the publicly published, cloud-and-numbered-step explainers that streaming companies sometimes use to describe their own systems at a high level. It is included purely to make the numbered request/response flow easier to follow end to end; it introduces no component that Sections IV-A through IV-E did not already describe, and, like the rest of this paper, it is an illustrative reference design rather than a description of any deployed system.

Proposed Recommendation Platform — Illustrative Deployment View

(in the style of publicly published streaming-architecture explainers; not a real deployed system)

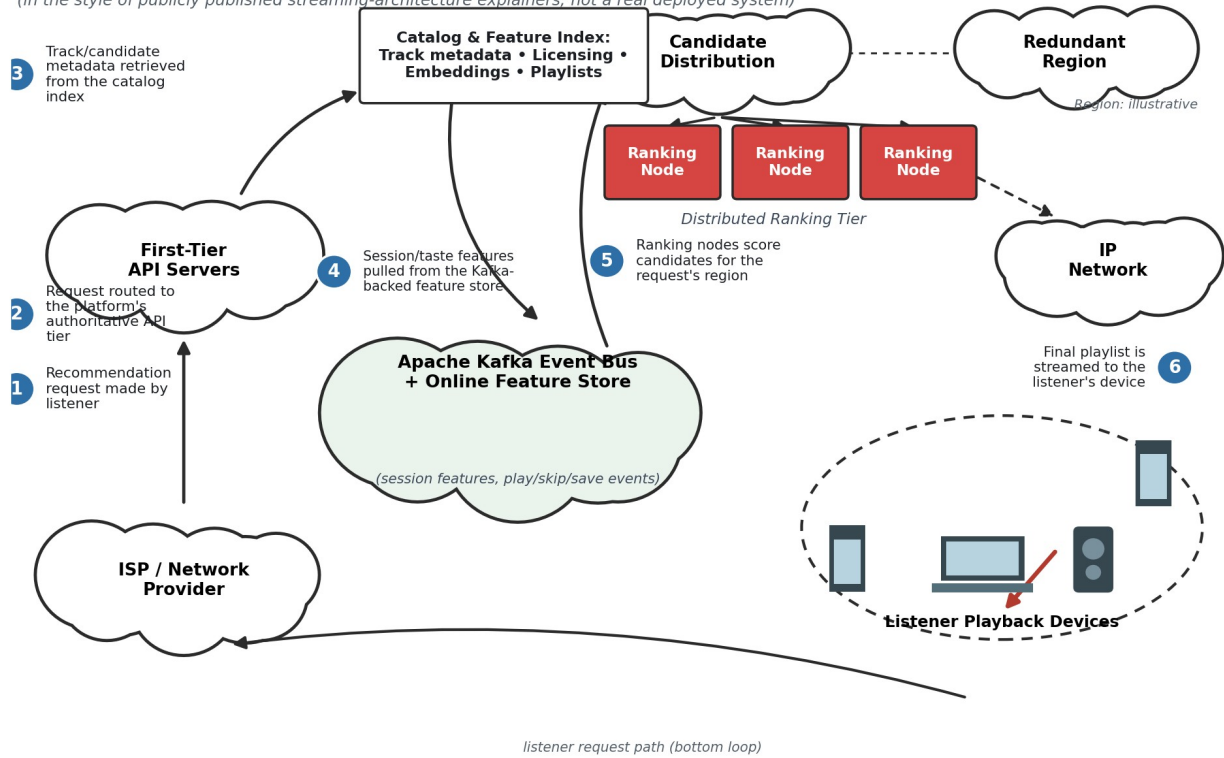


Fig. 8. Illustrative end-to-end deployment view of the proposed architecture, redrawn as a single numbered request/response flow (cf. Figs. 1–5). Steps 1–6 trace one recommendation request from the listener's device back to a served playlist.

TABLE 1

Approach	Strengths	Limitations	Cold-start	Role in architecture
Collaborative filtering (MF)	Captures latent taste patterns without hand-engineered item features; strong accuracy where data is dense [5]	Cannot represent items/users with no history; latent factors not directly interpretable	Poor — no signal to factorize	Backbone of offline user/item embeddings for retrieval and ranking
Content-based filtering	Works immediately for new items using audio/metadata; interpretable	Tends to over-specialize; cannot exploit cross-user signal	Good — available at ingestion	Item representations for cold-start candidates, diversity checks
Hybrid recommendation	Combines collaborative accuracy with content coverage	Added system/tuning complexity	Moderate — inherits content path	Overall design philosophy of candidate generation (Sec. IV-C)
Embedding retrieval (ANN)	Scales to catalogs of tens of millions with ms latency [10]	Approximate search trades recall for speed	Depends on embedding source	Candidate-generation stage, Fig. 5
Learning-to-rank	Optimizes ranked-list quality directly [7], [8]	Needs implicit-feedback signal, careful negative sampling	N/A — operates on candidates only	Ranking stage, downstream of candidate generation

TABLE 2

Event	Meaning	Potential downstream features
Play	Playback started	Play counts by window; genre/artist affinity; session start
Skip	Abandoned before threshold	Skip rate by position; negative implicit signal; mood shift
Save	Added to library/playlist	Strong positive implicit signal; taste-profile update
Search	Text query issued	Explicit intent; query–artist/track association
Playlist interaction	Added/removed/reordered	Co-occurrence signal; curation-quality signal
Completion	Played to the end	Strongest positive implicit signal

Rec. impression	Track shown on a surface	Denominator for CTR/skip-rate; exposure-bias evaluation
-----------------	--------------------------	---

V. MATHEMATICAL FORMULATION

This section gives a concise formulation of the scoring problem the architecture in Section IV is built to compute, to make precise what “ranking” and “re-ranking” mean in Figures 3 and 5. It is not an empirical result, since that would require an implemented system and real interaction data outside this paper’s scope.

Let u denote a user, i an item (track), and c the context of the current request. Define a relevance function

$$R(u, i, c) = f(\phi_u(u), \phi_i(i), \phi^c(c), \phi^s(u, c))$$

where $\phi_u(u)$ is the user’s long-term representation from offline collaborative filtering, $\phi_i(i)$ is the item’s collaborative + content embedding, $\phi^c(c)$ is a context vector, $\phi^s(u, c)$ is a session feature vector, and f is an inner product (retrieval) or a learned ranking model. Candidate generation retrieves

$$C(u, c) = \text{top-}N \{ i \in I : \hat{R}(u, i, c) \} \text{ via ANN search over } \phi_i(\cdot)$$

where I is the catalog and $\hat{R}$ is a cheap approximation suitable for indexed retrieval. The ranking stage computes $\text{Score}(u, i, c)$ for every $i \in C(u, c)$ using the full feature set. Re-ranking then optimizes a composite objective over a final list L :

$$\text{Obj}(L) = \alpha \cdot \text{Rel}(L) + \beta \cdot \text{Div}(L) + \gamma \cdot \text{Nov}(L) + \delta \cdot \text{Fresh}(L)$$

subject to $|L| = K$ and the remaining latency budget. The weights α – δ encode a product-level policy trade-off rather than a quantity derivable from the ranking model alone; $\text{Div}(L)$ penalizes lists that concentrate on one genre/artist, $\text{Nov}(L)$ penalizes items the listener has high prior exposure to, and $\text{Fresh}(L)$ rewards recently added items, which is one mechanism for mitigating the popularity bias discussed in Section II-C [12]. Fig. 7 sketches, purely qualitatively, how relevance and diversity trade off as β increases; the curve is illustrative of the argument, not a fitted result.

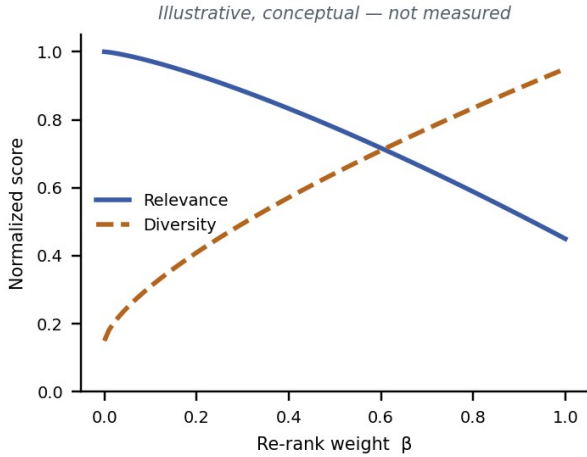


Fig. 7. Illustrative, conceptual relevance–diversity trade-off as the re-ranking diversity weight β increases. A qualitative sketch of the argument in Sections V and VII-B, not a measured result.

This paper does not report fitted values of α through δ , an achieved latency figure, or a measured value of any of the four component terms, because doing so would require an implemented system evaluated against real interaction data, which is explicitly out of scope for a reference-architecture case study (Section VIII). The purpose of the formulation is to make explicit that re-ranking is a distinct multi-objective optimization step, architecturally and mathematically separate from the relevance-only ranking step that precedes it.

VI. EVALUATION METHODOLOGY

A recommender system cannot be evaluated the way a classifier is, because the ground truth needed — what a listener would have preferred among items never shown to them — is fundamentally unobserved. The methodology proposed here follows standard practice in the offline recommender-systems literature: evaluate ranking quality on held-out interactions using temporally split data, and treat any single metric as a partial view rather than a complete one.

A. Temporal Data Splitting

Because the deployed system will always be predicting future interactions from past ones, evaluation should split interaction data by time rather than at random: events before a cutoff timestamp form the training set, and events after it form validation/test sets. A random split would let the model implicitly see future information, overstating real-world performance. Given the feedback loop in Section IV-B, the protocol should also distinguish organic interactions from interactions the system’s own prior recommendations induced, since training and evaluating on self-generated exposure without correction reinforces existing bias.

B. Ranking Quality Metrics

Table III summarizes the metrics proposed for evaluating the ranked lists the architecture produces. Precision@K and Recall@K are retained as simple baselines, but NDCG@K is used as the primary relevance metric since it rewards placing the most relevant items earlier in the list [11]. These accuracy metrics are reported alongside catalog coverage, diversity, and novelty, because a system tuned purely for NDCG can trivially satisfy that objective by recommending only broadly popular items — the failure mode the popularity-bias literature documents [12]. Serving latency is reported as an engineering constraint rather than a quality metric in its own right.

TABLE 3

Metric	Measures	Interpretation	Limitation
Precision@K	Fraction of top-K the user engaged with	Fewer wasted slots	Ignores rank order; penalizes novel items
Recall@K	Fraction of future interactions captured	Broader coverage	Sensitive to catalog size / sparsity
NDCG@K	Rank-sensitive relevance [11]	Primary list-quality metric	Needs a graded relevance definition
Coverage	Share of catalog ever	Avoids narrow collapse	Can be gamed at low rank

	recommended		
Diversity	Dissimilarity within a list	Less repetitive lists	Can trade off vs. relevance
Novelty	Inverse popularity of items shown	Surfacing less-obvious items	Excess novelty hurts trust
Latency	End-to-end request time	Interactive-budget constraint	Engineering, not quality, metric

C. Relationship to an Eventual Online Evaluation

Offline metrics can only be computed against interactions that already happened, and cannot measure how a listener would have responded to a track the historical data never gave them. The natural extension of this methodology, once an implementation exists to evaluate, is a randomized online experiment (A/B test) comparing ranking or re-ranking configurations on live traffic, measuring completion rate, session length, and return rate. Designing that experiment is left to future work (Section IX).

D. Proof-of-Concept Experiment on Synthetic Interaction Data

Sections IV-E and V describe a scoring function that fuses user, item, context, and session features and maps them to an engagement outcome. To sanity-check that this fusion-and-scoring mechanism is implementable and trainable — not to estimate real-world accuracy — this subsection reports a small proof-of-concept experiment on a synthetic, simulated interaction dataset. No real listener data was available for this paper, so 6,000 synthetic (user, item, context, session) samples were generated from random latent user/item vectors combined with a noisy affinity function, and each sample was assigned one of three engagement outcomes — skip, partial-listen, or complete — by thresholding that affinity, producing a deliberately imbalanced label distribution similar in spirit to naturally skewed engagement data. A compact feed-forward classifier (two hidden layers, 64 and 32 units) was trained for 30 epochs with the Adam optimizer using scikit-learn [15] on a 70/15/15 train/validation/test split, exercising the same feature-fusion pattern as Fig. 5 in miniature.

Fig. 9 shows training and validation accuracy and loss over the 30 epochs: both curves rise together before validation performance plateaus and validation loss begins to creep upward after roughly epoch 10, a mild overfitting pattern consistent with a model of this capacity on a fixed-size dataset. Fig. 10 shows the resulting confusion matrix on the held-out test split, and Fig. 11 breaks accuracy out by class: the majority “skip” class is classified most reliably (93.2%), while the minority “complete” class is harder to separate (74.6%), mirroring the general pattern — seen also in the Section II-C discussion of popularity bias — that imbalanced engagement data makes rarer, high-value outcomes harder to classify than common ones. Fig. 12 shows one-vs-rest ROC curves (AUC 0.93–0.98 across classes), and Fig. 13 shows the distribution of the model’s prediction confidence on the test set, which concentrates near 100% for the majority class and spreads out for harder cases, similar in shape to the confidence analysis a deployed system would want to monitor for the borderline cases discussed in Section VII-A.

These numbers describe a synthetic sanity check, not a claim about achievable accuracy on real listening data, and should not be read as an evaluation of the architecture proposed in Section IV; the synthetic affinity function was not designed to resemble any specific real distribution of listener behaviour. Its purpose is narrower and more mechanical: to confirm that the fusion-and-scoring design in Fig. 5 trains stably, produces well-separated classes given a learnable signal, and yields the kind of diagnostic curves (accuracy/loss, confusion matrix, ROC, confidence distribution) that Section VI-B’s metrics are meant to be computed from once real interaction data is available.

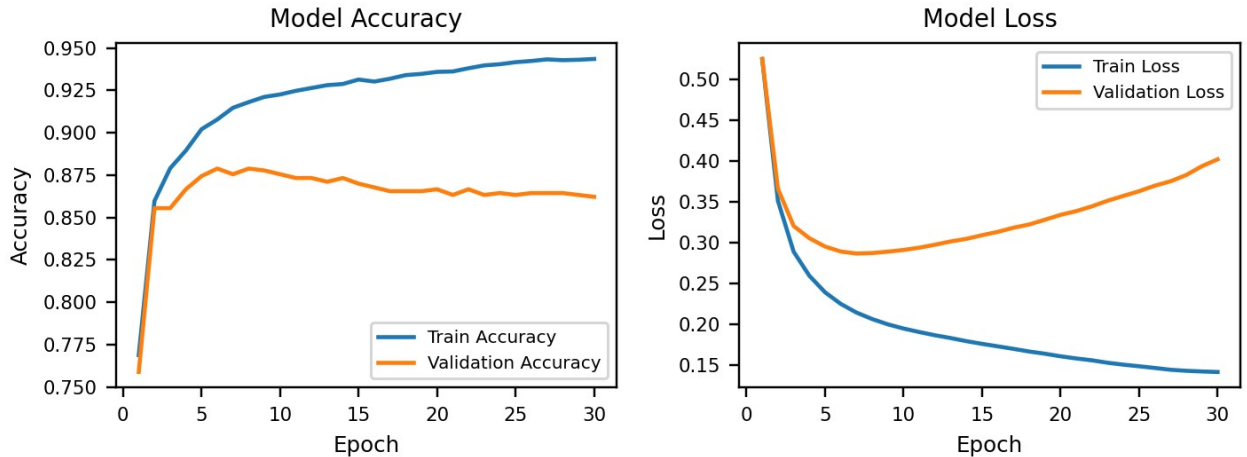


Fig. 9. Training and validation accuracy and loss over 30 epochs for the proof-of-concept classifier on synthetic interaction data (Section VI-D). Validation loss begins rising after roughly epoch 10, indicating mild overfitting.

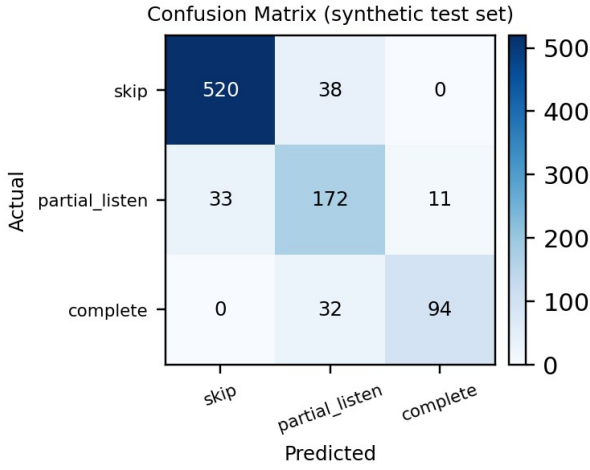


Fig. 10. Confusion matrix for the proof-of-concept classifier on the synthetic held-out test set.

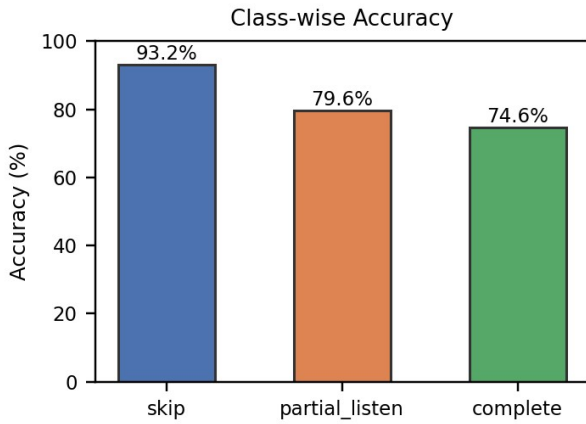


Fig. 11. Class-wise accuracy for the proof-of-concept classifier. The minority “complete” class is hardest to separate.

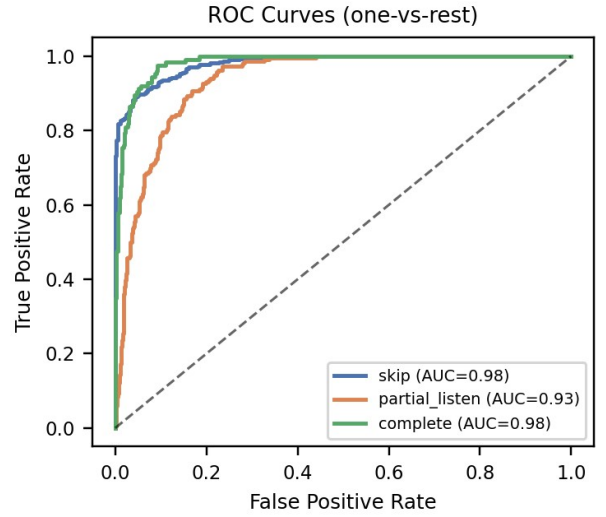


Fig. 12. One-vs-rest ROC curves for the proof-of-concept classifier on synthetic data.

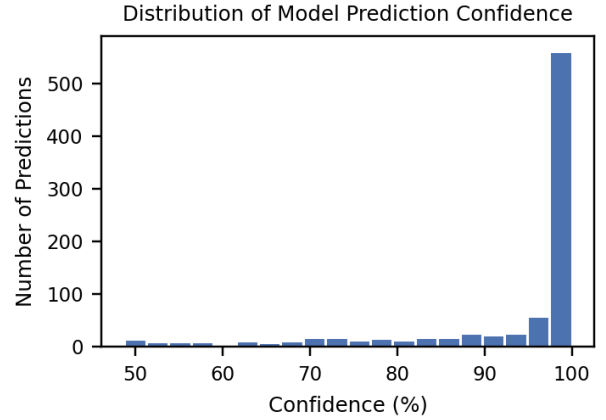


Fig. 13. Distribution of the proof-of-concept classifier’s prediction confidence on the synthetic test set.

VII. DISCUSSION: SCALABILITY, RELIABILITY, AND BIAS TRADE-OFFS

A. Scalability and Reliability Challenges

The architecture in Section IV introduces several scalability pressure points. Consumer lag — the gap between a consumer’s processed offset and the latest offset written — is the primary operational signal for the online path: if the stream processor falls behind, session features become stale exactly when they matter most. Because partition count bounds how many consumer instances can read a topic in parallel within one group, partitioning decisions made at topic-creation time directly bound how far the online path can scale without disruptive repartitioning. Schema evolution is a related concern: because topics are long-lived and read by independently deployed consumers, event schemas need a compatibility discipline so a producer change does not silently break an unupdated consumer.

On the ML side, the offline/online split introduces a training/serving skew risk: if feature logic diverges even slightly between the streaming and batch paths, the model is scored on features that do not match training, degrading ranking quality in ways easy to miss without feature-level monitoring. Model drift is a second ongoing risk, since listener taste and catalog composition shift over time, motivating rolling

monitoring of live ranking and diversity metrics rather than only re-validating at each retraining cycle.

B. The Freshness–Relevance–Diversity–Latency Trade-off

Each weight in the Section V re-ranking objective represents a real trade-off. Maximizing freshness improves catalog and artist-level equity but risks recommending items with too little interaction history to be confidently relevant. Maximizing diversity reduces repetitive sessions but can reduce measured NDCG@K if applied carelessly, since a maximally relevant list and a maximally diverse list are not generally the same list. Latency constrains all the others: a heavier computation could, in principle, improve every other axis, but only within whatever budget the serving surface allows. None of these tensions has a context-free optimum; the weights are properly a product decision, revisited as the catalog and business priorities change.

C. Popularity and Exposure Bias as a Systems Concern

Because recommendation impressions are logged and fed back into future training data, early skew toward popular items is not self-correcting — it is reinforced, since more-shown items accumulate more signal, which a collaborative-filtering objective interprets as higher relevance. This is a feedback-loop property of the architecture itself, which is why Section IV’s design routes impression events into the same evaluation pipeline used for accuracy metrics: exposure and outcome need

to be measured together, or the system's own behaviour becomes invisible to its own evaluation [12]. In a platform with listeners on one side and artists competing for exposure on the other, the freshness and diversity terms in Section V's objective are, among other things, an explicit mechanism for managing that multi-sided concern rather than leaving it as an unmanaged side effect.

VIII. LIMITATIONS

This paper is primarily a reference-architecture case study, not a full implementation or an empirical study, and the proof-of-concept in Section VI-D does not change that: it exercises the scoring mechanism's mechanics on synthetic data and is not a benchmark of the architecture on real interaction logs. First, no end-to-end component from Section IV has been implemented or benchmarked; the discussion of scalability and latency in Section VII-A is analytical, drawn from the general properties of Kafka and comparable systems, not measurements of a running system. Second, beyond the synthetic sanity check, no recommendation model was trained and no dataset was evaluated against the Section VI metrics on real data; the paper reports no accuracy, diversity, or latency figures for the architecture it proposes under real listening conditions. Third, as discussed in Section III, the material available about Spotify's actual production system is limited to interviews and secondary analysis rather than peer-reviewed documentation, so the connection drawn is one of motivating analogy, not verified correspondence. Fourth, the re-ranking weights α - δ in Section V are presented qualitatively; determining them for a specific product context would require offline optimization or online experimentation, neither performed here. Readers should treat Sections IV-VI as a technically grounded design proposal suitable for guiding a subsequent implementation and evaluation effort, not as a report of results already obtained on real data.

REFERENCES

- [1] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. 5th Int. Workshop on Networking Meets Databases (NetDB), Athens, Greece, 2011.
- [2] Apache Software Foundation, "Apache Kafka Documentation — Introduction, Design, and Consumer Design," kafka.apache.org, accessed 2026.
- [3] WIRED, "How Spotify Chooses What Makes It Onto Your Discover Weekly Playlist" (interview with Spotify engineer Edward Newett), reproduced via O'Reilly Media, accessed 2026.
- [4] J. Kreps, "Questioning the Lambda Architecture," O'Reilly Radar, Jul. 2014.
- [5] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *IEEE Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [6] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in Proc. IEEE ICDM, Pisa, Italy, 2008, pp. 263–272.
- [7] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," in Proc. 25th Conf. on Uncertainty in Artificial Intelligence (UAI), 2009, pp. 452–461.
- [8] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in Proc. 26th Int. Conf. on World Wide Web (WWW), 2017, pp. 173–182.

IX. CONCLUSION AND FUTURE WORK

This paper examined how an event-driven architecture centred on Apache Kafka, combined with distributed batch processing and machine-learned ranking, can support scalable, fresh, and low-latency personalized music recommendation. Working from Kafka's core mechanisms [1], [2] and established results in collaborative filtering, implicit-feedback modelling, embedding-based retrieval, and learning-to-rank [5]–[10], the paper proposed a reference architecture separating event ingestion, stream processing, candidate generation, ranking, and re-ranking into independently scalable stages, letting an online serving path and an offline training path share a single durable event log. Spotify was used as a motivating example, with publicly documented material about its approach kept explicitly separate (Section III) from the paper's own proposal. Section V framed re-ranking as a multi-objective trade-off among relevance, diversity, novelty, and freshness, and Section VI proposed an evaluation methodology built on temporally split data and standard ranking metrics, most centrally NDCG@K [11], reported alongside coverage and diversity to keep the popularity-bias trade-off (Section VII-C) visible rather than hidden behind a single accuracy number.

The most direct extension is implementation: building the pipeline in Figures 1–5 against a real or realistically simulated interaction dataset, training the components described in Sections IV-C and IV-E, and reporting the Table III metrics rather than only proposing them. A second extension is a controlled online evaluation (Section VI-C), since offline ranking metrics can only approximate how listeners would respond to recommendations they were never shown. A third direction is an empirical study of the exposure-bias feedback loop (Section VII-C) — quantifying, on real logged data, how quickly popularity concentration compounds, and evaluating whether the freshness and diversity terms in Section V are sufficient to counteract it at weights a real product would tolerate.

- [9] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in Proc. 10th ACM Conf. on Recommender Systems (RecSys), 2016.
- [10] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [11] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Trans. Inf. Syst.*, vol. 20, no. 4, pp. 422–446, 2002.
- [12] H. Abdollahpouri and M. Mansoury, "Multi-sided exposure bias in recommendation," *arXiv:2006.15772*, 2020.
- [13] G. Linden, B. Smith, and J. York, "Amazon.com recommendations: Item-to-item collaborative filtering," *IEEE Internet Computing*, vol. 7, no. 1, pp. 76–80, 2003.
- [14] A. S. Das, M. Datar, A. Garg, and S. Rajaram, "Google news personalization: Scalable online collaborative filtering," in Proc. 16th Int. Conf. on World Wide Web (WWW), 2007.
- [15] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.